

2020 시스템 프로그래밍
- Bomb Lab -

제출일자	2020.11.01
분 반	02
이 름	최재범
학 번	201702081

Phase 1 [결과 화면 캡처]

```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
I can see Russia from my house!
Phase 1 defused. How about the next one?
```

Phase 1 [진행 과정 설명]

rsi에다가 어떤 주소를 계산하고(=0x2b30) strings_not_equal 호출하는 루틴이 보임
이 주소를 그대로 따라가보면 정답이 나옴

Phase 1 [정답]

I can see Russia from my house!

Phase 2 [결과 화면 캡처]

```
0 1 1 2 3 5
That's number 2. Keep going!
```

Phase 2 [진행 과정 설명]

read_six_numbers

rsi = rip + 0x12c6 : 조사해보면, 포맷스트링 (입력 6개 정수)
scanf의 리턴값으로 나오는 eax와 0x5 비교, 작으면 터짐
-> 입력 6개

phase_2

(rsp)와 0 비교하여 같아야 함 -> 첫번째 인자 0
(rsp+4)와 1 비교하여 같아야함 -> 두번째 인자 1
+48 rbx = rsp -> 첫번째 인자 위치
+51 rbp = rbx + 0x10 -> 첫번째인자+0x10 위치

+66 eax = (rbx+4) -> 첫번째인자+4에 저장된 값 eax대입 -> 2번째인자
+69 eax = (rbx) + eax -> 2번째인자에 1번째 인자 더함
+71 eax와 (rbx+8)이 같아야 함 -> 3번째와, 1번째+2번째가 같아야함 -> 3번째 인자 1

+57 rbx = rbx + 4 -> rbx(포인터임)를 위치 4만큼 이동
+61 rbp, rbx 비교 -> rbp는 rbx 이동의 상한선
다르다 : goto 66 ~ 71
3번째인자 eax대입
3번째인자에 2번째인자 더함
4번째와, 2번째 + 3번째가 같아야함 -> 4번째 인자 2
-> n항 : n-2항 + n-1항 의 규칙

Phase 2 [정답]

0 1 1 2 3 5

Phase 3 [결과 화면 캡처]

```
5 4294966865
Halfway there!
```

Phase 3 [진행 과정 설명]

+28 : scanf 앞의 (0x1ab7+rip) 조사해보면, 포맷스트링 (입력 2개 정수)

explode_bomb : +74 +140 +208

74 : +40,43 ~ scanf 로 1개보다 많이 넣어야 함

208 : +45,49 ~ (rsp) 가 7보다 작거나 같아야 함 -> scanf 바로 뒤이므로 : 첫번째 인자값 <= 7

+55 eax = (rsp) -> 첫번째 인자값

+58 rdx = rip + 0x17ec -> 주소 (주소에 -6108 들어있음)

+65 rax = (rdx+4*rdx) -> rdx에 저장된 주소값에서 4*첫인자 만큼 이동한 곳에 있는 값

+69 rax = rdx + rax

--> 65~69 : rax = 2rdx+4rax

일단 5 5로 입력넣어도 문제없으니 140으로 넘어감

140 :

입력을 5 5로 넣어봄

입력 5 5일 시: 72->187->192->113->118->123->128->132->134->138 실행

+128 : 첫번째 인자값(rsp에 위치)이 5보다 작거나 같아야 함

+134 : 두번째 인자값(rsp+4에 위치)가 eax와 같아야 함 -> eax = 4294966865

따라서 5 4294966865 를 넣으면 됨

Phase 3 [정답]

5 4294966865

Phase 4 [결과 화면 캡처]

```
8 1
So you got that one. Try this one.
```

Phase 4 [진행 과정 설명]

+28 : scanf 앞의 (0x1997+rip) 조사해보면, 포맷스트링 (입력 2개 정수)

explode_bomb : +51 +86

51 : +40,43 ~ scanf로 2개를 넣어야 함

+45,49 ~ 첫번째 인자값(rsp에 위치)이 0xe(10진수 14)보다 작거나 같아야 함 (unsigned) -> 첫번째 인자 0, 14 사이

86 : +74,77 ~ eax가 0x1과 같아야 함

+79,84 ~ (rsp+4) 가 0x1과 같아야 함 -> 두번째 인자값 1

+56 edx = 0xe

+61 esi = 0x0

+66 edi = (rsp) -> 첫번째 인자값

+69 func4 호출 -> 첫번째 인자값, 0x0, 0xe가 인자로 들어감

-> 리턴값이 1이어야 함

입력을 0 1 ~ 14 1 까지 다 해봄 -> 8 1 성공

Phase 4 [정답]

8 1

Phase 5 [결과 화면 캡처]

```
5 115
Good work! On to the next...
```

Phase 5 [진행 과정 설명]

+28 : scanf 앞의 (0x1922+rip) 조사해보면, 포맷스트링 (입력 2개 정수)

explode_bomb : +109 +135

109 : +45 eax = (rsp) -> 첫번째 인자
+48 eax = eax & 0xf -> 하위 4비트만 가져옴
+51 (rsp) = eax -> 첫번째 인자를 하위 4비트만 남김
+54,57 eax 와 0xf 가 같으면 안됨 -> 첫번째 인자는 하위 4비트가 0xf(15)이면 안됨

+59 ecx = 0
+64 edx = 0
+69 rsi = rip+0x166c -> 배열 -> 검사해보면, 64바이트만큼의 배열이 있음을 알 수 있음

```
(gdb) x/32wd 0x55555556ba0
0x55555556ba0 <array.3417>: 10      2      14      7
0x55555556bb0 <array.3417+16>: 8       12     15     11
0x55555556bc0 <array.3417+32>: 0       4      1      13
0x55555556bd0 <array.3417+48>: 3       9      6      5
```

+76 edx = edx + 1 -> 반복 카운터, 나중에 검사
+81 eax = (rsi + rax * 4) -> rsi(배열)에서, rax*4만큼 이동한 위치의 값을 저장 -> 탐색 단위 4바이트

-> 16원소

+84 ecx = ecx + eax -> eax 누적, 나중에 검사
+86,89 eax 와 0xf 가 같지 않으면 +76으로 -> 원소가 15면 반복 종료함

+91 (rsp) = 0xf -> 첫번째 인자를 0xf로
+98,101 edx 와 0xf 가 같아야 함 -> +76에서 1씩 증가, 총 15번 탐색하여야 함
+103,107 ecx 와 (rsp+4) 가 같아야 함 -> ecx와 두번째 인자 같아야 함

원소가 15인 경우부터 시작하여 역순으로 15번 탐색해 나가면, 지나친 원소를 알 수 있음
=> 15 = arr[6] -> 6 = arr[14] -> 14 = arr[2] -> 2 = arr[1] -> 1 = arr[10] -> 10 = arr[0]
-> 0 = arr[8] -> 8 = arr[4] -> 4 = arr[9] -> 9 = arr[13] -> 13 = arr[11] -> 11 = arr[7]
7 = arr[3] -> 3 = arr[12] -> 12 = arr[5]

eax : 12 + 3 + 7 + 11 + 13 + 9 + 4 + 8 + 0 + 10 + 1 + 2 + 14 + 6 + 15
를 수행하고, 루프가 종료됨. 따라서 총 누적된 ecx는
0 ~ 15 중 5를 제외한 숫자를 전부 더한 것 = 115 -> 두번째 인자
첫번째 인자는 처음 탐색 인덱스인 5

135 :

+40,43 ~ scanf로 1개보다 많이 넣어야 함

Phase 5 [정답]

5 115

Phase 6 [결과 화면 캡처]

```
2 5 6 1 3 4
Congratulations! You've defused the bomb!
Your instructor has been notified and will verify your solution.
[Inferior 1 (process 18436) exited normally]
(gdb) █
```

Phase 6 [진행 과정 설명]

read_six_numbers : 6개 정수 입력

+42 r14d = 0

+48 goto +87

+57 ebx++ -> +65 상태의 카운터 + 1

+60,63 ebx > 5 이면 : goto +83

+65 rax = ebx -> +115 상태의 카운터 저장 (인덱스)

+68 eax = (rsp+rax*4) -> 다음 번째 인자 저장

+71,74 eax와 (rbp+0) 이 달라야 함 -> 현재 rbp : 인자 배열 시작주소
다르면 goto +57

+83 r13+=4 -> 첫번째 인자 주소 + 4함 -> 다음 번째 인자로 변경

+87 rbp = r13

+90 eax = (r13+0) -> r13에 저장된 값 : 검사하면 첫번째 인자

+94 eax-- -> 에서 1뺀

+97,100 eax <= 5이어야 함

+102 r14d++ -> 카운터 증가

+106,110 r14d == 6이면 : goto +117 -> 카운터 6 도달할 시 goto +117

+112 ebx = r14d -> 현재 카운터 저장

+115 goto +65

+48 : 처음에 카운터 0으로 초기화 후 +87로 점프

+87 : 첫번째 인자 <= 5 을 확인 후, 카운터 증가 후 +65로 점프

+65 : 첫번째 인자 != 현재 인자 를 확인, +57로 점프

+57 : 카운터 증가 후 계속 반복하며 첫번째 인자 != 나머지 인자 를 확인 / 카운터가 6에 도달하면 +83으로 점프

+83 : 탐색하는 인자를 다음번째 인자로 변경, 이후 +87부터 계속 반복

-> +83~115 는 모든 인자가 다 다를 것을 확인하는 루틴

+117 rcx = r12 + 0x18(10진수로 24) -> r12 를 검사해보면 인자 배열의 시작주소

+122 edx = 7

+127 eax = edx

+129 eax = eax - (r12)

+133 (r12) = eax

+137 r12 = r12 + 4

+141,144 r12, rcx

다르면 goto +127

+117 : rcx를 인자배열 뒤의 위치로 함
 +122 ~ 133 : 7에서 첫번째 인자값 뺀 것으로 덮어 씌움
 +137 ~ 144 : 모든 인자에 대하여 반복

```
+146 esi = 0
+151 goto +179

+153 rdx = (rdx + 8)
+157 eax++
+160,162 ecx != eax 이면 goto +153

+164 (rsp + rsi * 8) + 0x20(10진수32) = rdx
+169 rsi++
+173,177 rsi == 6이면 goto +201

+179 ecx = (rsp+rsi*4)
+182 eax = 1
+187 rdx = rip + 0x3be3    -> node1 시작 주소
+194,197 ecx 가 1보다 크면 goto +153
+199 goto +164
```

(입력 : 1 2 3 4 5 6)

+146 ~ 199 :

explode로 가는 루틴이 없기 때문에, 스킵하고 201까지 넘기고 rsp 근처 변화만 관찰함

node 1을 조사하면, (고유값 : 4바이트 / 1 ~ 6 / 다음 노드 주소) 의 16바이트 구조 -> 5개의 노드

node 5에 적혀있는 주소를 따라가면, node 6까지 발견 (0x555555758110) -> 총 6개의 노드 발견

이 노드들은 링크드 리스트를 나타냄

```
(gdb) x/32x 0x555555758220
0x555555758220 <node1>: 0x0000015e    0x00000001    0x55758230    0x00005555
0x555555758230 <node2>: 0x000001dc    0x00000002    0x55758240    0x00005555
0x555555758240 <node3>: 0x000000a1    0x00000003    0x55758250    0x00005555
0x555555758250 <node4>: 0x000000e1    0x00000004    0x55758260    0x00005555
0x555555758260 <node5>: 0x00000200    0x00000005    0x55758110    0x00005555
0x555555758270: 0x00000000    0x00000000    0x00000000    0x00000000

(gdb) x/16x 0x555555758110
0x555555758110 <node6>: 0x000000e8    0x00000006    0x00000000    0x00000000
```

결과로, rsp+32부터 노드 주소로 보이는 6개의 값이 생겼음 (총 7개지만, 하나는 1번문제의 답이 들어있는 다른 것)

이는 7에서 노드 번호(입력값 순서) 를 뺀 것과 동일한 순서로 되어있음

```
(gdb) x/32x $rsp+0x20
0x7fffffff3c0: 0x55758110    0x00005555    0x55758260    0x00005555
0x7fffffff3d0: 0x55758250    0x00005555    0x55758240    0x00005555
0x7fffffff3e0: 0x55758230    0x00005555    0x55758220    0x00005555
0x7fffffff3f0: 0x557586c0    0x00005555    0x95150000    0xc3d6248c
0x7fffffff400: 0x00000000    0x00000000    0x55556960    0x00005555
0x7fffffff410: 0x55555060    0x00005555    0xffff510    0x00007fff
0x7fffffff420: 0x00000000    0x00000000    0x55555260    0x00005555
0x7fffffff430: 0x00000000    0x00000000    0xf7a05b97    0x00007fff
```

+201 ~ 251 :

+201 부터는 rsp 근처에 저장된 노드 주소에 접근하여, 범용 레지스터를 이용하여 계속 값을 swap하고있음
swap이 끝나는 +264에 bp걸고 노드 값의 변화를 봄 -> 순서가 1-2-3-4-5-6 에서, 6-5-4-3-2-1 로 변화

```
(gdb) x/32x 0x555555758220
0x555555758220 <node1>: 0x0000015e      0x00000001      0x00000000      0x00000000
0x555555758230 <node2>: 0x000001dc      0x00000002      0x55758220      0x00005555
0x555555758240 <node3>: 0x000000a1      0x00000003      0x55758230      0x00005555
0x555555758250 <node4>: 0x000000e1      0x00000004      0x55758240      0x00005555
0x555555758260 <node5>: 0x00000200      0x00000005      0x55758250      0x00005555
0x555555758270: 0x00000000      0x00000000      0x00000000      0x00000000
```

+259 : 카운터 5로 설정

+264 : rbx에 변화 후의 첫 노드 들어있음

+275, 279 : rbx 다음 노드에 위치한 고유값 저장

+281, 283 : rbx의 고유값이, 다음 노드의 고유값보다 더 크거나 같아야 함
goto +266

+266 ~ 273 : 다음 노드로 이동, 카운터 1 감소 / 카운터 0 될 때까지 반복

-> 첫노드부터 시작하여, 다음 노드의 고유값과 비교하여 내림차순이 되어야 함

(입력 : 1 3 5 2 4 6)

규칙성을 찾기 위해 입력을 바꾸어, +201,264 에 bp 걸고 다시 해봄

rsp 근처에는 7 - 입력값, 그리고 그 순서대로 노드의 주소가 적혀있음. -> 6 4 2 5 3 1 순
노드도 똑같이 6 4 2 5 3 1 로 배열됨

```
Breakpoint 14, 0x0000555555555683 in phase_6 ()
(gdb) x/32x $rsp
0x7fffffff3a0: 0x00000006      0x00000004      0x00000002      0x00000005
0x7fffffff3b0: 0x00000003      0x00000001      0x00000000      0x00000000
0x7fffffff3c0: 0x55758110      0x00005555      0x55758250      0x00005555
0x7fffffff3d0: 0x55758230      0x00005555      0x55758260      0x00005555
0x7fffffff3e0: 0x55758240      0x00005555      0x55758220      0x00005555
0x7fffffff3f0: 0x557586c0      0x00005555      0x2fe1be00      0x807d7aff
0x7fffffff400: 0x00000000      0x00000000      0x55556960      0x00005555
0x7fffffff410: 0x55555060      0x00005555      0xfffffe510      0x00007fff
```

```
(gdb) x/32x 0x555555758220
0x555555758220 <node1>: 0x0000015e      0x00000001      0x00000000      0x00000000
0x555555758230 <node2>: 0x000001dc      0x00000002      0x55758260      0x00005555
0x555555758240 <node3>: 0x000000a1      0x00000003      0x55758220      0x00005555
0x555555758250 <node4>: 0x000000e1      0x00000004      0x55758230      0x00005555
0x555555758260 <node5>: 0x00000200      0x00000005      0x55758240      0x00005555
0x555555758270: 0x00000000      0x00000000      0x00000000      0x00000000
```

따라서, 7-(입력값) 의 순서가, 노드 고유값 기준으로 내림차순이 되어야 하므로

고유값은 0x200(5) > 0x1dc(2) > 0x15e(1) > 0x0e8(6) > 0x0e1(4) > 0x0a1(3)

-> 7-입력값 순서 : 5 2 1 6 4 3

-> 입력값 순서 : 2 5 6 1 3 4

Phase 6 [정답]

2 5 6 1 3 4